MongoDB

annumathur003@gmail.com

APNA COLLEGE

The Shell

mongosh //to start

use college //to create & use a new database called "college"

```
Last login: Sun Apr 12 09:32:08 on console
gauravmathur@Annus-Mac ~ % mongosh
Current Mongosh Log ID: 69de05dfa0c3c87f5d79a64a
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSele
ctionTimeoutMS=2000&appName=mongosh+2.7.0
MongoNetworkError: connect ECONNREFUSED 127.0.0.1:27017
gauravmathur@Annus-Mac ~ % brew services start mongodb-community
✔ JSON API cask_tap_migrations.jws.json         Downloaded    2.4KB/  2.4KB
✔ JSON API formula_tap_migrations.jws.json      Downloaded    2.0KB/  2.0KB
✔ JSON API cask.jws.json                        Downloaded   15.4MB/ 15.4MB
✔ JSON API formula.jws.json                     Downloaded   32.0MB/ 32.0MB
==> Successfully started `mongodb-community` (label: homebrew.mxcl.mongodb-
gauravmathur@Annus-Mac ~ % mongosh
Current Mongosh Log ID: 69de07235fb666ce3ba196a5
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSele
ctionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:          8.2.6
Using Mongosh:          2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-04-14T14:48:04.587+05:30: Access control is not enabled for the database. R
ead and write access to data and configuration is unrestricted
------

test> 1+2
3
test> "annumathur".toUppercase()
TypeError: "annumathur".toUppercase is not a function
test> "annumathur".toUpperCase()
ANNUMATHUR
test>
```

```
test> show dbs
admin   40.00 KiB
config  80.00 KiB
local   64.00 KiB
test> use college
switched to db college
```

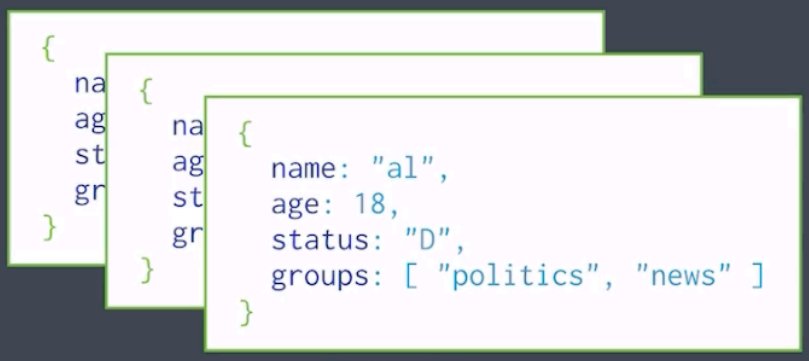
JSON vs BSON

	JSON	BSON
Encoding	UTF-8 String	Binary
Data Support	String, Boolean, Number, Array, Object, null	String, Boolean, Number (Integer, Float, Long, Decimal128...), Array, null, Date, BinData
Readability	Human and Machine	Machine Only

Collections

Document : Mongo stores data in form of documents (BSON docs)

Collection : MongoDB stores documents in collections.



Collection

```
college> db.student.insertOne( {name:"Annu" , age:21 , marks:91} )
{
  acknowledged: true,
  insertedId: ObjectId('69de0ce85fb666ce3ba196a6')
}
college> db.student.find()
[
  {
    _id: ObjectId('69de0ce85fb666ce3ba196a6'),
    name: 'Annu',
    age: 21,
    marks: 91
  }
]
college>
```

INSERT in DB

insertOne

`db.collection.insertOne()`	Inserts a single document into a collection.

show collections

db.student.insertOne({ name: “adam”, marks: 79 })

db.student.find( )

If a collection does not exist, MongoDB creates the collection when you first store data for that collection.

```
college> db.student.insertMany([{name:"Annu Mathur" , marks:89},{name:"Sandhya", marks:95} ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69de0ed35fb666ce3ba196a7'),
    '1': ObjectId('69de0ed35fb666ce3ba196a8')
  }
}
college>
```

INSERT in DB

insertMany (array of documents)

db.student.insertMany([{ name: “bob”, marks: 65 }, { name: “eve”, city: “Delhi“ }])

`db.collection.insertMany()`	Inserts multiple documents into a collection.

```
college> db.student.find()
[[
  {
    _id: ObjectId('69de0ce85fb666ce3ba196a6'),
    name: 'Annu',
    age: 21,
    marks: 91
  },
  {
    _id: ObjectId('69de0ed35fb666ce3ba196a7'),
    name: 'Annu Mathur',
    marks: 89
  },
  {
    _id: ObjectId('69de0ed35fb666ce3ba196a8'),
    name: 'Sandhya',
    marks: 95
  }
]
college> 
```

annumathur003@gmail.com

APNA COLLEGE

city : Delhi

FIND in DB

db.collection.find( ) //returns everything

for specific queries

db.collection.find({ key: value }) → cursor (reference to original)

db.collection.findOne({ key: value }) → actual document

FIND in DB

Query Operators

Q. Find students where marks > 75

```
db.student.find( {marks: {$gt: 75}} )
```

Q. Find students who live in Delhi or Mumbai

```
db.student.find( {city: {$in: ["Delhi", "Mumbai"]}} )
```

Q. Find students who scored > 75 or live in Delhi

```
db.student.find( {$or: [ {marks: {$gt: 75}}, {city: "Delhi"}]} )
```

annumathur003@gmail.com

UPDATE in DB

updateOne

`db.collection.updateOne()`	Updates at most a single document that match a specified filter even though multiple documents may match the specified filter.

numathur003@gmail.com

db.collection.updateOne(<filter> , <update>, <options>)

```
db.student.updateOne( {name:"adam"}, {$set: {marks:99}} )
```